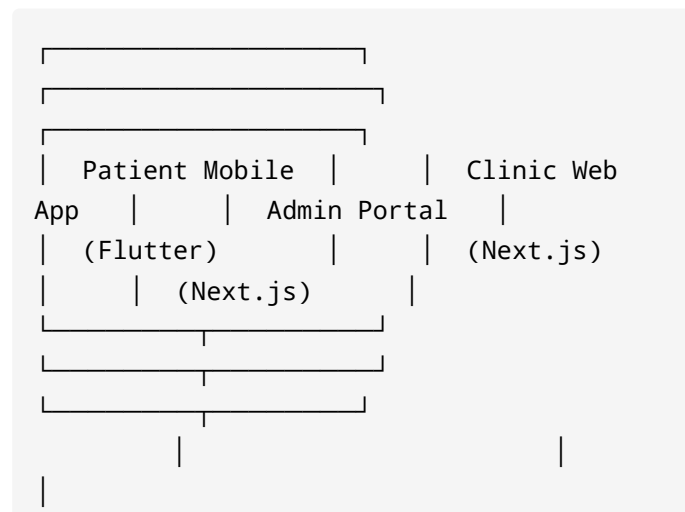


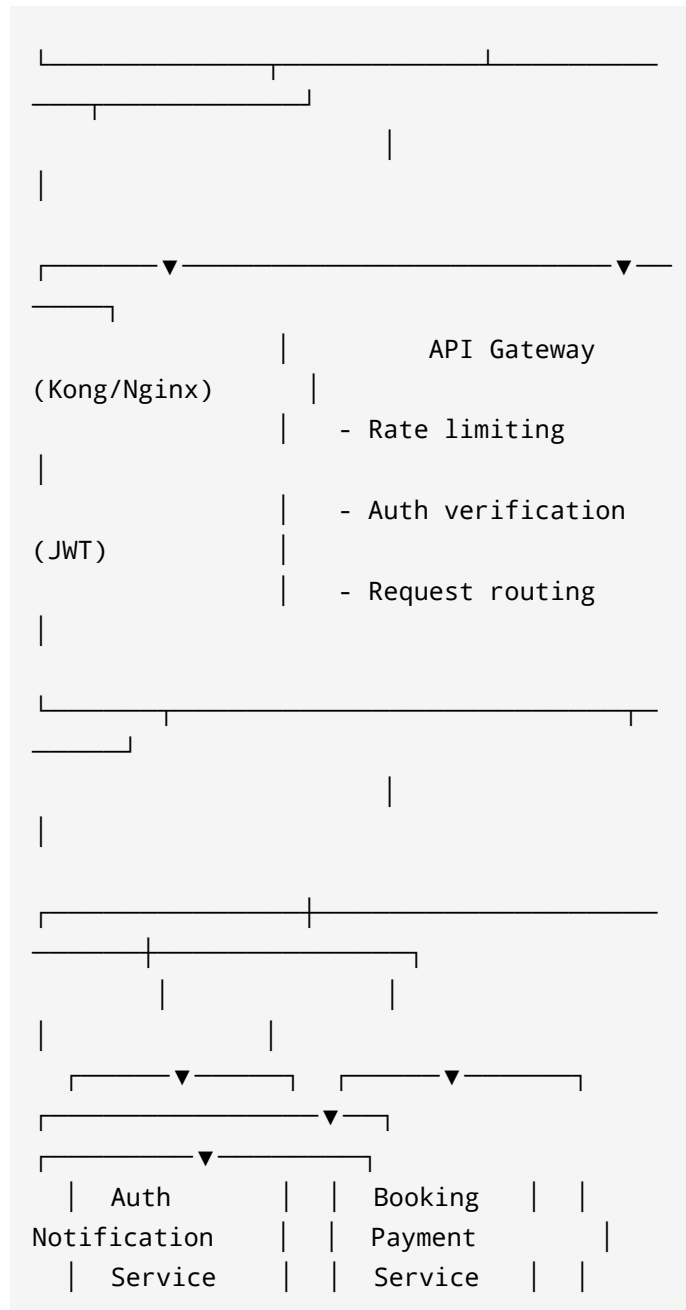
معمارية منصة حجز العيادات (Doctolib-style) – Clinic Booking Platform

1. نظرة عامة على النظام (System Overview)

عيادة وحدة: **Multi-tenant SaaS** المنصة مبنية على مبدأ معزول منطقياً tenant خاص فيها، وكل tenant تسجل، تحصل (row-level isolation) لكن يشارك نفس البنية التحتية

(High-level Components) المكونات الرئيسية





في البداية → تقسمه لخدمات مستقلة لما **Modular Monolith** يكبر الحمل فعلياً.

يعني:

- واضحة modules كود واحد، لكن مقسم داخلياً ل (auth/, booking/, notifications/, payments/)
- بين ال queries لا تخط له حدوده الواضحة module كل (modules)
- يحتاج booking service مثلاً) لما يوصل حمل معين تفصله لخدمة منفصلة، (مستقل scale

full بدأت فعلياً — مو Doctolib هذا بالضبط شلون شركات مثل microservices من اليوم الأول.

3. Tech Stack المقترح

الطبقة	التقنية	السبب
Patient Mobile App	Flutter	واحد ل codebase وأنت عندك، iOS/Android، خبرة فيه من مشاريعك السابقة
Clinic Web Dashboard	Next.js 14 (App Router) + TypeScript	لو تبي SEO، للسرعة SSR صفحات هبوط للعيادات

الطبقة	التقنية	السبب
Backend API	Node.js (NestJS) أو Supabase Edge Functions	NestJS = بنية enterprise واضحة (DI, modules, guards)
Database	PostgreSQL	علاقات معقدة (مرضى / — مواعيد/أطباء/عيادات) NoSQL أفضل من SQL هنا
Cache/Queue	Redis	rate limiting, session cache
Realtime	Supabase Realtime أو WebSockets	تحديث فوري لحالة الموعد (confirmed/cancelled)
Auth	Supabase Auth أو Auth0	من الصفر أبداً auth لا تبني — مخاطرة أمنية غير ضرورية
Payments	ZainCash / Qi Card / Stripe (لو (توسعت دولياً	حسب خبرتك بمشروع الـ proxy purchasing

الطبقة	التقنية	السبب
Notifications	Firebase Cloud Messaging + WhatsApp Business API	feature تذكير المواعيد أهم no-show يقلل
File Storage	Supabase Storage / S3	صور الأشعة، التقارير الطبية
Infra	Docker + CI/CD (GitHub Actions)	،حتى لو صغير المشروع من اول يوم يوفر docker عليك وجع راس لاحقاً

4. Database Schema (المخطط)

(الأساسي)

الجداول الجوهرية

```
-- Multi-tenancy: كل عيادة = tenant
clinics (
  id UUID PK,
  name TEXT,
  subdomain TEXT UNIQUE,    -- clinic-
                             name.yourapp.com
```

```

    address TEXT,
    phone TEXT,
    subscription_tier TEXT, --
free/pro/enterprise
    created_at TIMESTAMPTZ
)

-- المستخدمين (نظام موحد للأطباء والسكرتارية)
staff (
    id UUID PK,
    clinic_id UUID FK -> clinics,
    role TEXT, -- 'doctor' | 'receptionist'
| 'admin'
    full_name TEXT,
    specialty TEXT,
    auth_user_id UUID FK -> auth.users
)

-- عابرين لكل العيادات، staff منفصلين عن المرضى
patients (
    id UUID PK,
    auth_user_id UUID FK -> auth.users,
    full_name TEXT,
    phone TEXT UNIQUE,
    date_of_birth DATE,
    national_id TEXT NULLABLE
)

-- أوقات العمل المتاحة لكل طبيب
doctor_availability (
    id UUID PK,
    staff_id UUID FK -> staff,

```

```

    day_of_week INT,
    start_time TIME,
    end_time TIME,
    slot_duration_minutes INT DEFAULT 30
)

-- المواعيد (الجدول الأهم بالنظام)
appointments (
    id UUID PK,
    clinic_id UUID FK -> clinics,
    patient_id UUID FK -> patients,
    staff_id UUID FK -> staff,
    scheduled_at TIMESTAMPTZ,
    duration_minutes INT,
    status TEXT, -- pending | confirmed |
completed | cancelled | no_show
    reason TEXT,
    created_at TIMESTAMPTZ,
    cancelled_reason TEXT NULLABLE
)

-- (مبسّط EHR لو راح تسوي) السجل الطبي
medical_records (
    id UUID PK,
    patient_id UUID FK -> patients,
    appointment_id UUID FK -> appointments,
    notes TEXT,
    attachments JSONB, -- روابط الملفات بال
storage
    created_by UUID FK -> staff
)

```

```
-- المدفوعات
payments (
  id UUID PK,
  appointment_id UUID FK -> appointments,
  amount_iqd NUMERIC,
  provider TEXT, -- zaincash | qicard |
cash
  status TEXT, -- pending | paid |
failed | refunded
  transaction_ref TEXT
)
```

DB: Row-Level Security أهم قرار معماري بالـ (RLS)

هذا يمنع عيادة clinic_id. لازم يكون معزول بـ query كل
توصل لبيانات عيادة ثانية بالغلط. لو تستخدم

Supabase/Postgres:

```
CREATE POLICY clinic_isolation ON
appointments
  USING (clinic_id =
current_setting('app.current_clinic_id')::
uuid);
```

data leaks هذا مو تفصيل بسيط — هذا خط الدفاع الأول ضد
.بين العيادات، وأغلب المشاريع الناشئة تتجاهله وتندم لاحقاً

أصعب جزء تقني (Booking Logic ال 5) (بالمشروع)

خطيرة race conditions الحجز يبدو بسيط لكنه فيه

.بنفس الثانية slot المشكلة: مريضين يحاولون يحجزون نفس ال

الحل الصحيح:

```
-- unique constraint على (staff_id, scheduled_at) استخدم
ALTER TABLE appointments
  ADD CONSTRAINT unique_slot UNIQUE
    (staff_id, scheduled_at)
  WHERE status != 'cancelled';

-- + Database transaction مع SELECT ...
FOR UPDATE
BEGIN;
SELECT * FROM appointments
  WHERE staff_id = $1 AND scheduled_at =
    $2
  FOR UPDATE;
-- insert لو ما لى صف، اعمل
COMMIT;
```

شوف الموعد ("application-level check لا تعتمد فقط على نفسه يمنع ال database لازم ال — ("فاضي ثم احجزه بهذا senior ومطور junior هذا الفرق بين مطور. duplicate. النوع من الأنظمة

6. عملي REST نمط) API Design ال 6.

```
POST    /api/v1/auth/register
POST    /api/v1/auth/login

GET      /api/v1/clinics/:id/doctors
GET      /api/v1/doctors/:id/availability?
date=2026-07-20

POST     /api/v1/appointments          # حجز
موعد
GET      /api/v1/appointments/:id
PATCH   /api/v1/appointments/:id/cancel
PATCH   /api/v1/appointments/:id/confirm
# من طرف العيادة

GET      /api/v1/patients/:id/history

POST     /api/v1/payments/initiate
POST     /api/v1/webhooks/zaincash    #
callback من مزود الدفع
```

نقاط مهمة:

- **Versioning** لأنك أكيد راح تغيير — (/v1/) من اول يوم القديم API ال لاحقاً وما تريد تكسر التطبيق القديم ال
- **Idempotency keys** الدفع والحجز، endpoints على على عشان لو انقطع النت وأعاد المستخدم الطلب، ما يصير حجز مكرر أو دفع مكرر.

نقطة حرجة — (Security) الأمان 7.

بالمجال الطبي

1. **التشفير:** بيانات المرضى الحساسة (national_id, medical notes) تُشفّر at-rest (pgcrypto أو application-level encryption).
 2. **Audit log:** أي وصول لسجل مريض لازم يتسجل (مين شاف شنو ومتى) — هذا قانوني بأغلب الأنظمة الصحية وتوقعه (بأمريكا، وموازيها بمناطق أخرى HIPAA) العالمية. ينزل بأي تدقيق مستقبلي.
 3. **واضح:** دكتور يشوف مرضاه بس، سكرتيرة تشوف RBAC العيادة يشوف admin، الجدول بس بدون التفاصيل الطبية الكل.
 4. **Rate limiting** (login, booking) الحساسة endpoints على abuse. لمنع
-

8. (Roadmap) خطة التنفيذ المرحلية

(واقعي لمطور/فريق صغير)

Phase 1 — MVP (شهرين تقريباً)

- Auth (patient + clinic staff)
- CRUD للعيادات والأطباء
- race-condition handling بدون الحجز الأساسي (متقدم بعد)

- تطبيق موبايل بسيط: تصفح عيادات → حجز → إشعار تأكيد

Phase 2 — Production-ready

- Row-level security كامل
- (ZainCash/Qi Card) نظام الدفع
- (يقلل) قبل الموعد بـ 24 ساعة SMS/WhatsApp تذكيرات (بشكل كبير no-show)
- كامل للعيادة (تقويم، تقارير، إدارة Web dashboard الموظفين)

Phase 3 — Scale & Differentiate

- (EHR) سجل طبي إلكتروني مبسط
- تحليلات للعيادة (معدل الحضور، الإيرادات، أكثر الأطباء طلباً)
- للعيادات الكبيرة multi-branch دعم
- عام لو تريد تفتح تكامل مع أنظمة ثالثة API

أهم نصيحة معمارية واحدة 9.

لا تفكر بـ **Modular Monolith + Postgres + RLS** ابدأ بـ microservices, Kubernetes, أو event-driven architecture من اليوم الأول — هذا تعقيد ما تحتاجه إلا لو عندك آلاف العيادات فعلاً. الشركات الناجحة بهذا المجال

بدأت بسيطة وبنية واضحة، وبعدين (نفسها بالبداية Doctolib)
تطورت البنية مع نمو الحمل الفعلي — مو قبل